

Projet de Sécurité Informatique



"Signature Numérique Simple"

Réalisé par :

Marwa Maqsoudi — 23512157

Safaa Mounkid — 23502171

Année Universitaire 2025-2026

Table des matières

1	Introduction	3
2	Contexte et Problématique	4
3	Objectifs du projet	6
3.1	Comprendre la cryptographie asymétrique	6
3.2	Implémenter l'algorithme RSA	6
3.3	Garantir l'intégrité des messages	7
3.4	Concevoir une interface graphique conviviale	7
3.5	Manipuler les bibliothèques Python de cryptographie	7
4	Présentation de la Signature Numérique	8
4.1	Définition de la signature numérique	8
4.2	Principes fondamentaux	8
4.2.1	Authentification	8
4.2.2	Intégrité	8
4.2.3	Non-répudiation	8
4.3	Fonctionnement général	8
4.3.1	Étape 1 : Génération des clés	8
4.3.2	Étape 2 : Signature du message	9
4.3.3	Étape 3 : Vérification	9
4.4	Algorithmes utilisés	10
4.4.1	RSA	10
4.4.2	SHA-256	10
4.4.3	Padding PSS	10
5	Technologies et Outils Utilisés	11
5.1	Python	11
5.2	Tkinter	11
5.3	Cryptography	11
5.4	SHA-256	12
5.5	Pillow	12
5.6	Visual Studio Code	12
6	Architecture Générale du Projet	13
6.1	Module Interface Graphique	13
6.2	Module Cryptographique	13
6.3	Module de Stockage	13
6.4	Fonctionnement global du système	14

7	Explication du Fonctionnement	15
7.1	Génération des clés RSA	15
7.2	Signature du message	15
7.3	Vérification de la signature	15
8	Description Détaillée du Code	16
8.1	Importation des bibliothèques	16
8.2	Gestion des couleurs	16
8.3	Gestion des événements (Hover)	16
8.4	Fonction generate_keys()	16
8.5	Fonction sign_message()	16
8.6	Fonction verify_signature()	17
8.7	Interface graphique	17
9	Interface Graphique	18
9.1	Composants de l'interface	18
9.2	Boutons disponibles	18
9.3	Design de l'interface	19
10	Tests et Résultats	20
10.1	Test de génération des clés	20
10.2	Test de signature	20
10.3	Test de vérification correcte	21
10.4	Test après modification du message	21
11	Sécurité et Limites du Projet	23
11.1	Sécurité du système	23
11.2	Limites du projet	23
11.3	Conclusion sur la sécurité	23
12	Conclusion	24

1 Introduction

La sécurité des informations est devenue un élément essentiel dans les systèmes informatiques modernes. Avec l'évolution rapide des technologies numériques et l'augmentation des échanges de données via Internet, plusieurs menaces apparaissent, notamment

- la modification des données,
- l'usurpation d'identité,
- le vol d'informations,
- la falsification des messages.

Dans ce contexte, les techniques cryptographiques jouent un rôle fondamental dans la protection des communications numériques et des données sensibles. Parmi ces techniques, la signature numérique constitue une solution efficace permettant de garantir :

- l'intégrité des données
- l'authenticité de l'expéditeur
- la non-répudiation.

Le principe de la signature numérique repose sur la cryptographie asymétrique, notamment l'algorithme RSA. Cette méthode utilise une paire de clés :

- une clé privée utilisée pour signer le message
- une clé publique permettant de vérifier la validité de la signature.

Dans le cadre de ce projet, nous avons développé un système en Python permettant de simuler un mécanisme simple de signature numérique à travers la génération des clés RSA, la signature d'un message et la vérification de la signature. Le système possède également une interface graphique moderne réalisée avec Tkinter afin de faciliter l'interaction avec l'utilisateur et rendre le système plus intuitif.

2 Contexte et Problématique

Aujourd'hui, les échanges électroniques occupent une place très importante dans différents domaines tels que :

- les banques,
- le commerce électronique,
- les administrations,
- les réseaux sociaux,
- la messagerie électronique
- les systèmes cloud.

Cependant, les données échangées sur les réseaux peuvent être exposées à plusieurs menaces de sécurité. Un attaquant peut modifier un message, falsifier l'identité de l'expéditeur ou intercepter des informations sensibles.

Par exemple :

- un contrat électronique peut être modifié après son envoi
- un utilisateur malveillant peut envoyer un faux message en usurpant l'identité d'une autre personne.

La problématique principale de ce projet est donc la suivante :

Comment vérifier qu'un message reçu provient réellement de l'expéditeur et qu'il n'a subi aucune modification pendant la transmission ?

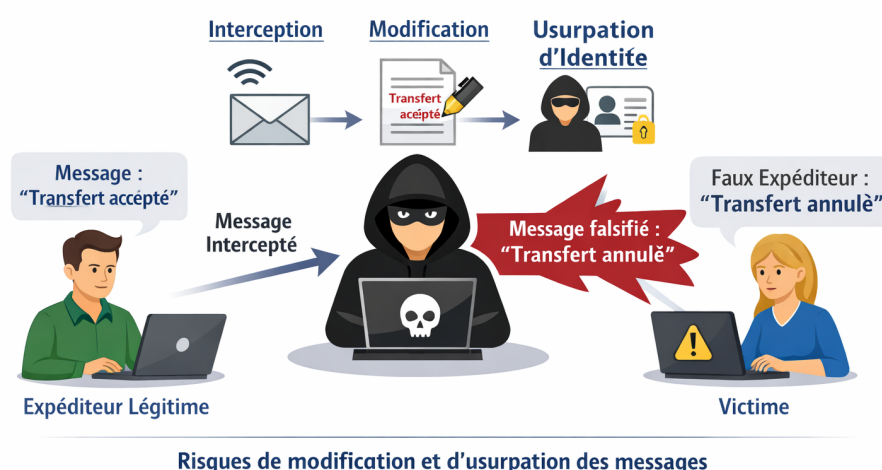


FIGURE 1 – Risques de modification et d'usurpation des messages

Pour résoudre ce problème, les systèmes modernes utilisent la signature numérique. Elle permet de :

- vérifier l'identité du signataire,
- détecter toute modification du contenu
- assurer la sécurité des échanges numériques.

Dans ce projet, nous avons simulé ce mécanisme en utilisant l'algorithme RSA et la fonction de hachage SHA-256.

3 Objectifs du projet

Le projet a été réalisé dans le but de comprendre et appliquer les notions fondamentales de la sécurité informatique et de la cryptographie.

Les objectifs principaux sont les suivants :

3.1 Comprendre la cryptographie asymétrique

L'objectif principal est d'étudier le fonctionnement de la cryptographie à clé publique, notamment :

- la notion de clé privée ;
- la notion de clé publique ;
- leur rôle dans la signature numérique.

3.2 Implémenter l'algorithme RSA

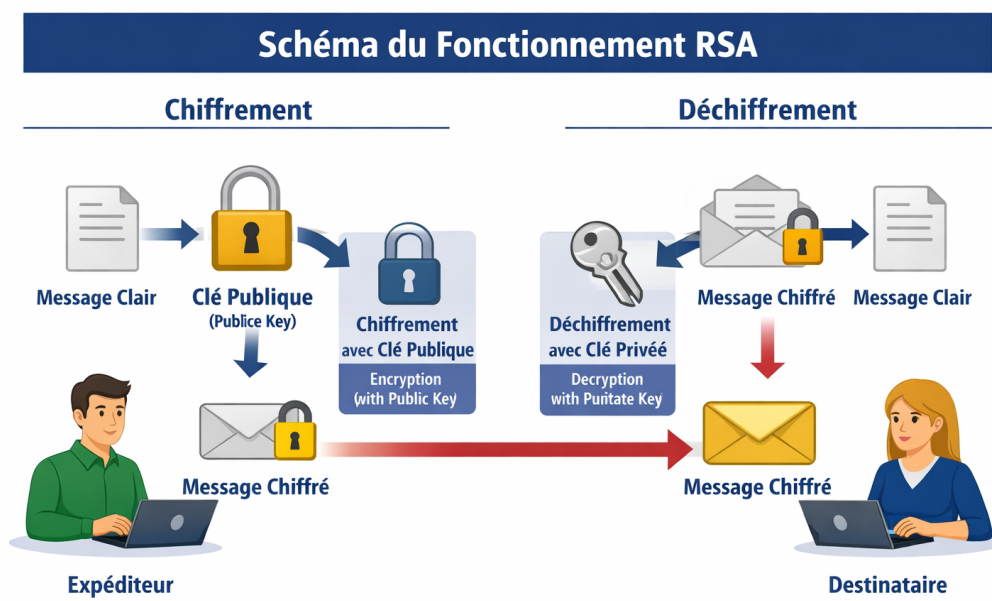


FIGURE 2 – Schéma du fonctionnement RSA : clé publique et clé privée

Le projet utilise l'algorithme RSA afin de :

- générer les clés cryptographiques ;
- signer les messages ;
- vérifier les signatures.

RSA est l'un des algorithmes les plus utilisés dans les systèmes de sécurité modernes.

3.3 Garantir l'intégrité des messages

Le système doit être capable de détecter toute modification du message après la signature.

Si le contenu du message change, la vérification doit échouer.

3.4 Concevoir une interface graphique conviviale

Une interface graphique simple et moderne a été développée avec Tkinter afin de :

- faciliter l'utilisation du programme ;
- améliorer l'expérience utilisateur ;
- rendre le projet plus interactif.

3.5 Manipuler les bibliothèques Python de cryptographie

Le projet permet également de découvrir l'utilisation de bibliothèques professionnelles comme :

- Tkinter ;
- Cryptography.

4 Présentation de la Signature Numérique

4.1 Définition de la signature numérique

La signature numérique est une technique cryptographique utilisée pour sécuriser les échanges électroniques. Elle permet de garantir :

- l'authenticité de l'expéditeur ;
- l'intégrité du message ;
- la non-répudiation.

Elle fonctionne de manière similaire à une signature manuscrite, mais sous une forme électronique sécurisée.

4.2 Principes fondamentaux

La signature numérique repose sur trois propriétés essentielles :

4.2.1 Authentification

Elle permet de confirmer l'identité de l'expéditeur. Le destinataire peut vérifier que le message provient bien de la bonne personne.

4.2.2 Intégrité

Elle garantit que le message n'a subi aucune modification pendant la transmission. Le moindre changement rend la signature invalide.

4.2.3 Non-répudiation

Le signataire ne peut pas nier avoir signé le message, car seule sa clé privée permet de générer la signature.

4.3 Fonctionnement général

Le processus de signature numérique se déroule en trois étapes :

4.3.1 Étape 1 : Génération des clés

L'utilisateur génère une paire de clés :

- une clé privée (secrète) ;
- une clé publique (partagée).

4.3.2 Étape 2 : Signature du message

Le message est d'abord haché avec SHA-256 :

$$H(m) = \text{SHA-256}(m)$$

Ensuite, le hachage est signé avec la clé privée RSA en utilisant le padding PSS. Le résultat est une signature numérique unique.

4.3.3 Étape 3 : Vérification

Le destinataire utilise la clé publique pour vérifier la signature.

Si la signature est valide :

- le message est authentique ;
- le message n'a pas été modifié.

Sinon, la signature est invalide.

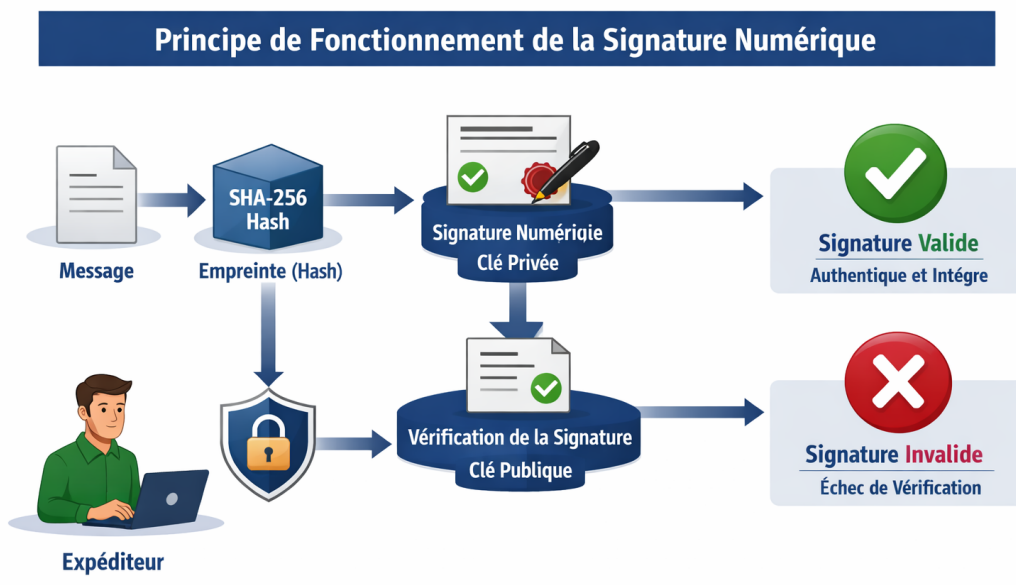


FIGURE 3 – Principe de fonctionnement de la signature numérique

4.4 Algorithmes utilisés

4.4.1 RSA

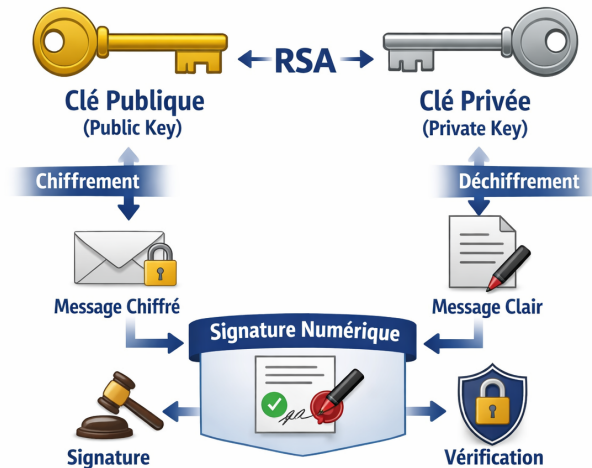


FIGURE 4 – Principe de l’algorithme RSA

RSA est un algorithme de cryptographie asymétrique inventé par Rivest, Shamir et Adleman. Il est largement utilisé dans :

- les signatures numériques ;
- le chiffrement ;
- les certificats SSL/TLS.

Dans ce projet, une taille de clé de 2048 bits est utilisée.

4.4.2 SHA-256

SHA-256 est une fonction de hachage cryptographique qui transforme un message de taille variable en une empreinte fixe de 256 bits. Elle est :

- rapide ;
- sécurisée ;
- résistante aux collisions.

4.4.3 Padding PSS

Le padding PSS (Probabilistic Signature Scheme) améliore la sécurité des signatures RSA en ajoutant des données aléatoires, rendant les attaques cryptographiques plus difficiles.

5 Technologies et Outils Utilisés

Plusieurs technologies ont été utilisées pour la réalisation de ce projet de signature numérique.

5.1 Python

Python est le langage principal utilisé dans ce projet. Il a été choisi pour sa simplicité, sa lisibilité et la richesse de ses bibliothèques dédiées à la cybersécurité et à la cryptographie.

5.2 Tkinter

Tkinter est la bibliothèque graphique standard de Python. Elle permet de créer une interface utilisateur comprenant :

- des fenêtres ;
- des boutons ;
- des champs de texte ;
- des messages d'erreur.

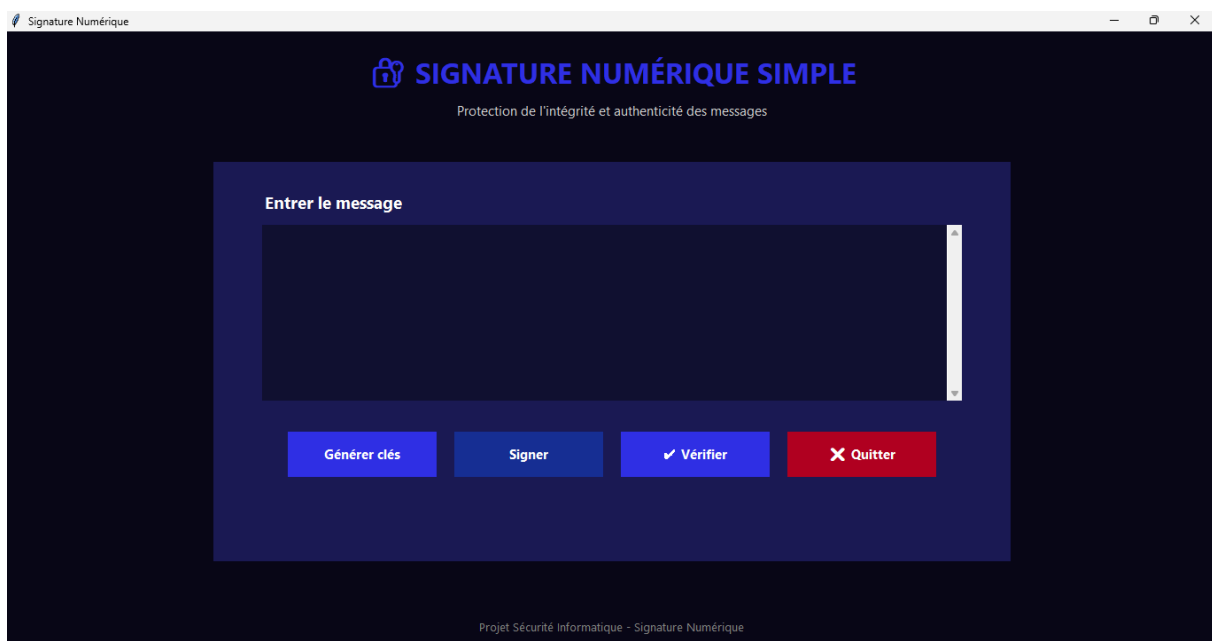


FIGURE 5 – Interface graphique de l'application Tkinter

5.3 Cryptography

La bibliothèque Cryptography est utilisée pour la sécurité et la cryptographie. Elle fournit :

- la génération de clés RSA ;
- les fonctions de hachage ;
- les mécanismes de signature numérique ;
- la sérialisation des clés.

5.4 SHA-256

SHA-256 est une fonction de hachage cryptographique utilisée pour transformer un message en une empreinte unique de 256 bits, garantissant l'intégrité des données.

5.5 Pillow

La bibliothèque Pillow a été utilisée pour la gestion des images et des icônes dans l'interface graphique.

5.6 Visual Studio Code

Visual Studio Code est l'environnement de développement utilisé pour coder et tester l'application. Il facilite :

- l'écriture du code ;
- le débogage ;
- l'exécution du programme.

6 Architecture Générale du Projet

Le système développé est organisé en plusieurs modules fonctionnels permettant de séparer les responsabilités et de faciliter la maintenance du code.

6.1 Module Interface Graphique

Ce module gère l'interaction avec l'utilisateur. Il est responsable de :

- la fenêtre principale de l'application ;
- les boutons d'action ;
- les événements utilisateur ;
- l'affichage des messages et résultats.

6.2 Module Cryptographique

Ce module implémente les opérations de sécurité et de cryptographie. Il permet :

- la génération des clés RSA ;
- la signature numérique des messages ;
- la vérification des signatures.

6.3 Module de Stockage

Ce module assure la sauvegarde locale des données générées. Les fichiers utilisés sont :

- **private_key.pem** : contient la clé privée ;
- **public_key.pem** : contient la clé publique ;
- **signature.bin** : contient la signature numérique.

6.4 Fonctionnement global du système

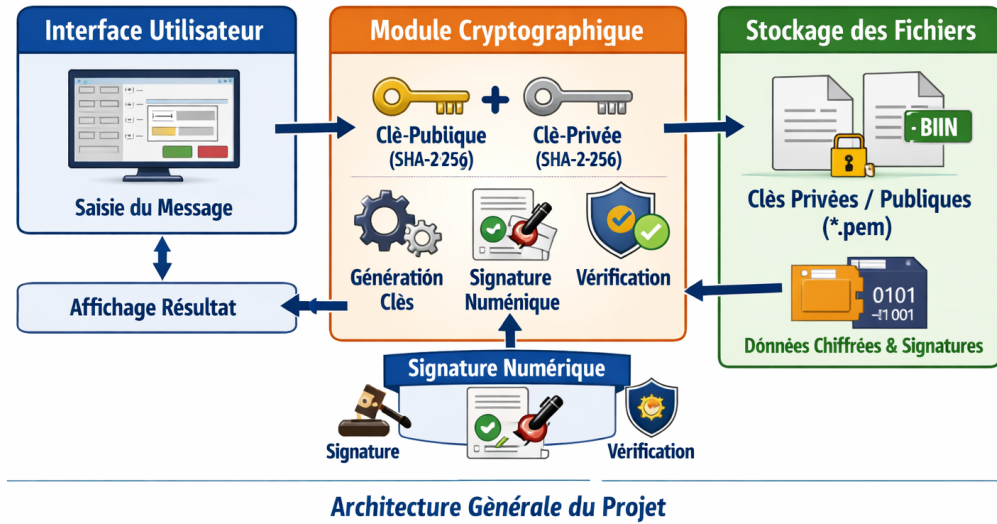


FIGURE 6 – Architecture globale du système

Le système suit le flux suivant :

Utilisateur → Saisie du message → Signature numérique → Vérification → Résultat

Ainsi, le message est signé à l'aide de la clé privée, puis vérifié avec la clé publique afin de garantir son intégrité et son authenticité.

7 Explication du Fonctionnement

Le fonctionnement du système de signature numérique est basé sur trois étapes principales : la génération des clés, la signature du message et la vérification de la signature.

7.1 Génération des clés RSA

La fonction `generate_keys()` permet de générer une paire de clés RSA.

Étapes :

- création d'une clé privée RSA ;
- génération de la clé publique associée ;
- sauvegarde des clés dans des fichiers `.pem`.

Paramètres utilisés :

- taille de clé : 2048 bits ;
- exposant public : 65537.

7.2 Signature du message

La fonction `sign_message()` permet de signer un message.

Étapes :

- récupération du message saisi par l'utilisateur ;
- chargement de la clé privée ;
- application de la fonction de hachage SHA-256 ;
- signature avec l'algorithme RSA-PSS ;
- sauvegarde de la signature dans le fichier `signature.bin`.

7.3 Vérification de la signature

La fonction `verify_signature()` permet de vérifier l'authenticité d'un message signé.

Étapes :

- lecture du message ;
- chargement de la clé publique ;
- lecture de la signature enregistrée ;
- comparaison cryptographique entre le message et la signature.

Résultat :

- Signature VALIDE si le message est authentique ;
- Signature INVALIDE si le message a été modifié.

8 Description Détaillée du Code

8.1 Importation des bibliothèques

Le programme commence par l'importation des bibliothèques nécessaires.

- `tkinter` : utilisé pour créer l'interface graphique ;
- `messagebox` : permet d'afficher des messages d'erreur ou d'information ;
- `ScrolledText` : fournit une zone de texte avec défilement.

Les modules de cryptographie sont également importés :

- `rsa` : génération des clés RSA ;
- `padding` : sécurisation des signatures ;
- `hashes` : implémentation de SHA-256 ;
- `serialization` : sauvegarde et chargement des clés.

Ces bibliothèques permettent d'assurer les fonctionnalités cryptographiques du projet.

8.2 Gestion des couleurs

Le programme définit plusieurs couleurs personnalisées afin de créer une interface graphique moderne au style sombre. Cela améliore l'expérience utilisateur et la lisibilité de l'application.

8.3 Gestion des événements (Hover)

Les fonctions `on_enter()` et `on_leave()` permettent de modifier dynamiquement la couleur des boutons lorsque l'utilisateur passe la souris dessus, ce qui améliore l'interactivité de l'interface.

8.4 Fonction `generate_keys()`

Cette fonction permet de générer une paire de clés RSA (privée et publique) et de les sauvegarder dans des fichiers `.pem`.

8.5 Fonction `sign_message()`

Cette fonction réalise les opérations suivantes :

- lecture du message saisi ;
- chargement de la clé privée ;
- application de la fonction SHA-256 ;
- génération de la signature numérique ;
- sauvegarde de la signature dans un fichier binaire `signature.bin`.

8.6 Fonction `verify_signature()`

Cette fonction permet de vérifier l'authenticité du message en comparant la signature enregistrée avec le message actuel.

8.7 Interface graphique

L'interface utilisateur est développée avec Tkinter. Elle contient :

- une zone de saisie pour le message ;
- des boutons d'action (génération, signature, vérification) ;
- des effets visuels (hover) ;
- une présentation moderne et ergonomique.

9 Interface Graphique

L'application développée dispose d'une interface graphique réalisée avec Tkinter afin de faciliter l'interaction avec l'utilisateur et rendre le système plus intuitif.

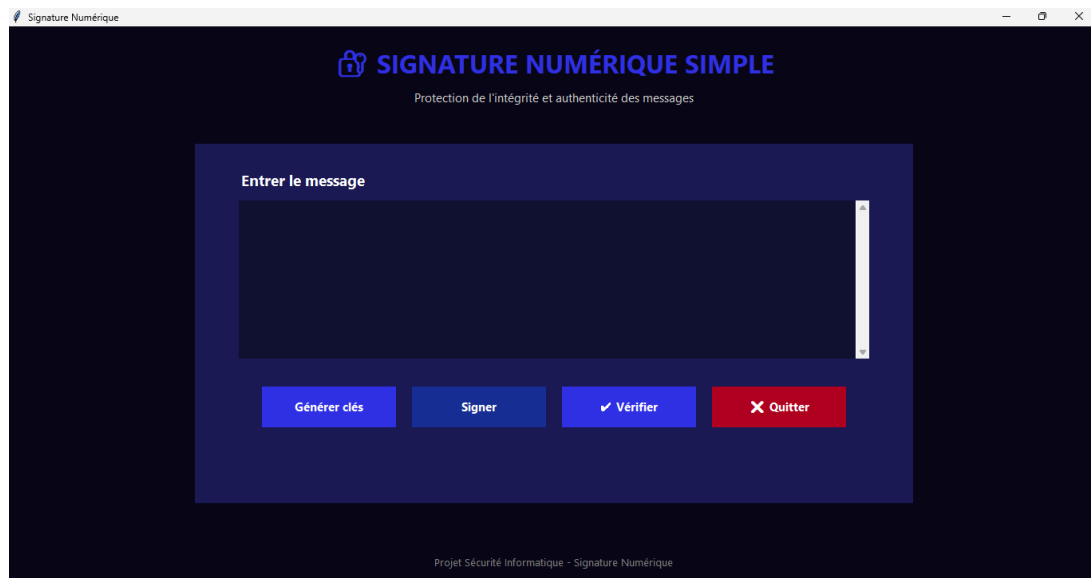


FIGURE 7 – Application de signature numérique en Python

9.1 Composants de l'interface

L'interface graphique contient les éléments suivants :

- un titre principal ;
- une description du projet ;
- une zone de texte pour la saisie du message ;
- plusieurs boutons interactifs ;
- un label d'affichage des résultats.

9.2 Boutons disponibles

Les principaux boutons de l'application sont présentés dans le tableau suivant :

Bouton	Fonction
Générer clés	Créer les clés RSA
Signer	Signer le message
Vérifier	Vérifier la signature
Quitter	Fermer l'application

9.3 Design de l'interface

L'interface a été conçue avec une palette de couleurs sombre afin de donner un style moderne et professionnel orienté cybersécurité.

Des effets interactifs (hover) ont été ajoutés sur les boutons afin d'améliorer l'expérience utilisateur.

Les icônes et éléments visuels contribuent également à rendre l'application plus intuitive et agréable à utiliser.

10 Tests et Résultats

Plusieurs tests ont été réalisés afin de vérifier le bon fonctionnement du système de signature numérique.

10.1 Test de génération des clés

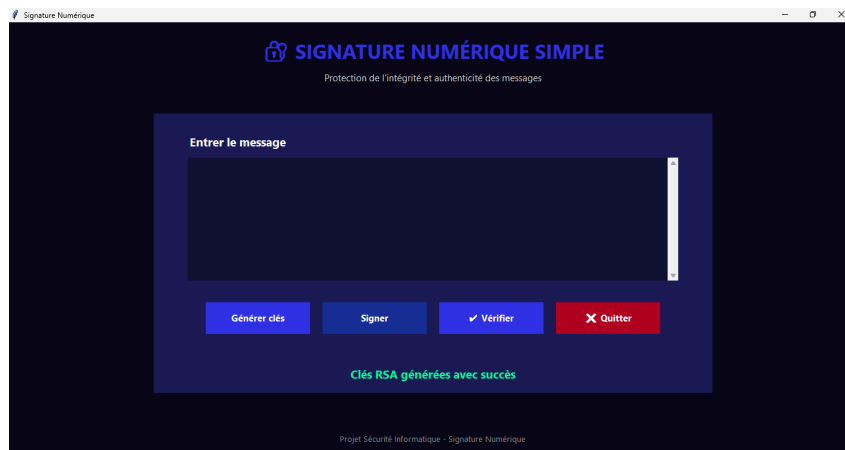


FIGURE 8 – Génération des clés RSA

Résultat :

- Les fichiers `private_key.pem` et `public_key.pem` sont créés correctement.

10.2 Test de signature

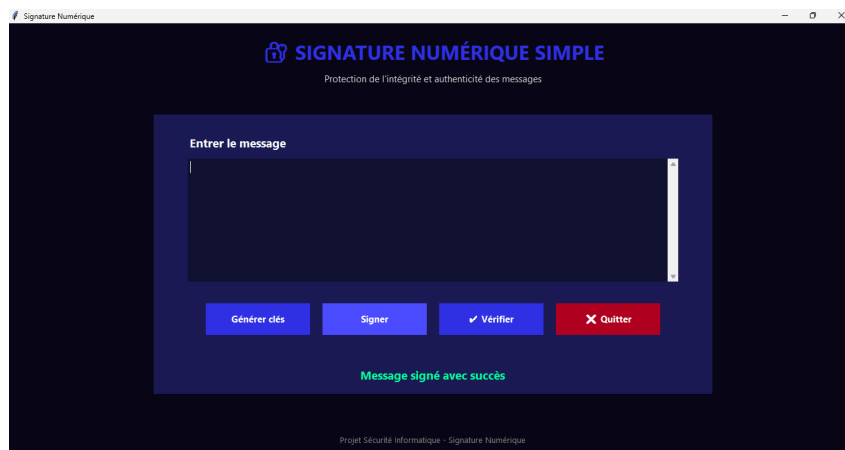


FIGURE 9 – Signature du message

Résultat :

- La signature est générée avec succès ;
- La signature est sauvegardée dans le fichier `signature.bin`.

10.3 Test de vérification correcte

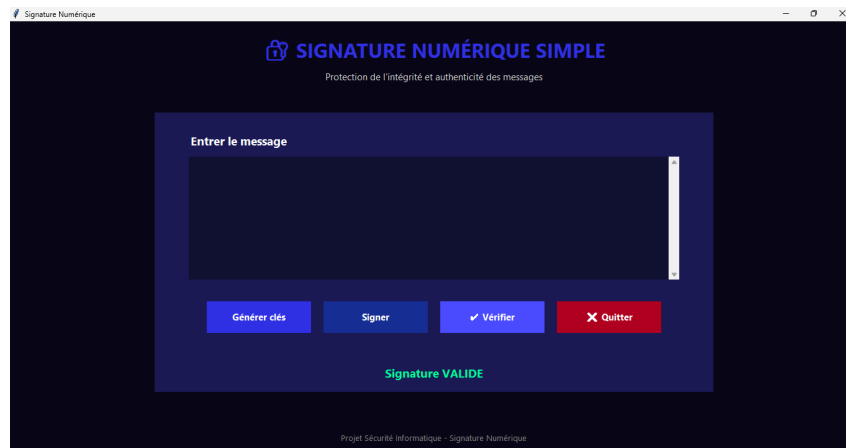


FIGURE 10 – Résultat de la vérification de la signature

Entrée :

— Message : Bonjour

Résultat :

— Signature VALIDE.

Le système confirme que le message est authentique et n'a pas été modifié.

10.4 Test après modification du message

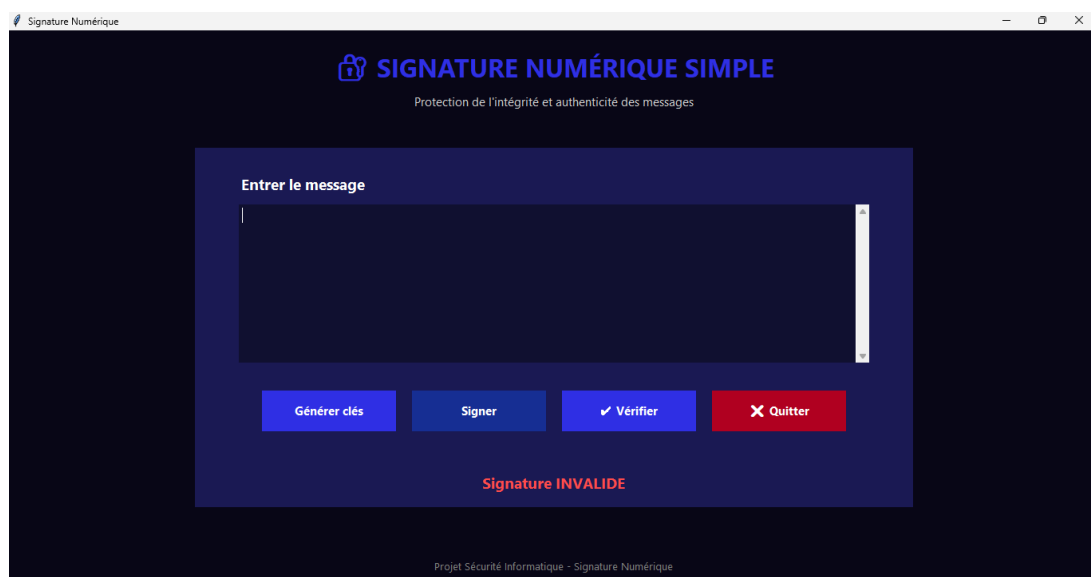


FIGURE 11 – Résultat de la vérification de la signature

Entrée :

- Message original : Bonjour
- Message modifié : Bonsoir

Résultat :

- Signature INVALIDE.

Le système détecte correctement toute modification du message, ce qui confirme l'efficacité du mécanisme de signature numérique.

11 Sécurité et Limites du Projet

11.1 Sécurité du système

Le système développé repose sur des mécanismes cryptographiques fiables permettant de garantir un certain niveau de sécurité. Les principaux éléments utilisés sont :

- l'algorithme RSA avec une taille de clé de 2048 bits ;
- la fonction de hachage SHA-256 ;
- un mécanisme de signature numérique sécurisé ;
- une interface utilisateur avec gestion des erreurs.

Ces éléments permettent de garantir l'intégrité et l'authenticité des messages.

11.2 Limites du projet

Malgré ses avantages, le système présente certaines limites :

- le projet reste une simulation pédagogique ;
- aucune gestion des certificats numériques ;
- absence de protection par mot de passe de la clé privée ;
- stockage des clés uniquement en local ;
- absence de communication réseau sécurisée ;
- application limitée à des messages texte simples.

11.3 Conclusion sur la sécurité

Malgré ces limites, ce projet constitue une bonne introduction aux mécanismes de sécurité utilisés dans les systèmes cryptographiques modernes et permet de comprendre les principes fondamentaux de la signature numérique.

12 Conclusion

Ce projet nous a permis de mettre en pratique plusieurs notions importantes de la sécurité informatique, notamment la cryptographie asymétrique et les signatures numériques.

Grâce à l'utilisation de l'algorithme RSA et de la fonction de hachage SHA-256, nous avons développé une application capable de signer et de vérifier des messages tout en garantissant leur authenticité et leur intégrité.

Les résultats obtenus montrent que le système fonctionne correctement et qu'il est capable de détecter efficacement toute modification des données.

Ce projet constitue ainsi une introduction concrète aux technologies de sécurité utilisées dans les systèmes modernes tels que les banques, les plateformes de commerce électronique et les communications sécurisées.